arXiv:2402.11131v1 [cs.CL] 16 Feb 2024

Speculative Streaming: Fast LLM Inference without Auxiliary Models

Nikhil Bhendawade [1] **Irina Belousova** [1] **Qichen Fu** [1] **Henry Mason** [1] **Mohammad Rastegari** [1] **Mahyar Najibi** [1]

Abstract

Speculative decoding is a prominent technique to speed up the inference of a large target language model based on predictions of an auxiliary draft model. While effective, in application-specific settings, it often involves fine-tuning both draft and target models to achieve high acceptance rates. As the number of downstream tasks grows, these draft models add significant complexity to inference systems. We propose Speculative Streaming, a single-model speculative decoding method that fuses drafting into the target model by changing the fine-tuning objective from next token prediction to future n-gram prediction. Speculative Streaming speeds up decoding by 1.8 - 3.1X in a diverse set of tasks, such as Summarization, Structured Queries, and Meaning Representation, without sacrificing generation quality. Additionally, Speculative Streaming is parameter-efficient. It achieves on-par/higher speed-ups than Medusa-style architectures while using ∼10000X fewer extra parameters, making it well-suited for resource-constrained devices.

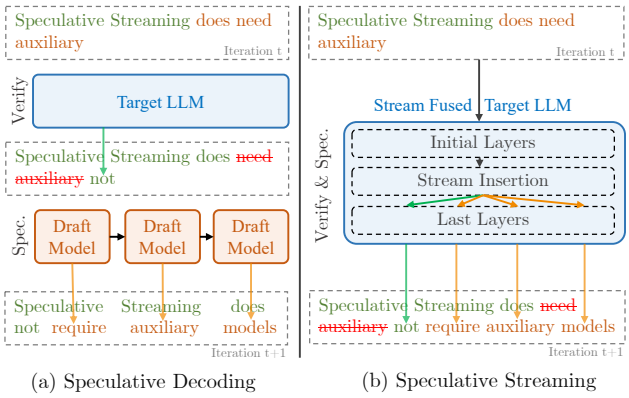


Figure 1. (a) Speculative Decoding requires a separate draft model that runs autoregressively to speculate. (b) Speculative Streaming significantly simplifies the system by performing speculation and verification concurrently, all within a single stream-fused model.

1. Introduction

Large transformers are today's preeminent tool for language modeling. The quality of these models improves as they scale (Kaplan et al., 2020), leading to the introduction of the state-of-the-art multi-billion parameter models (Brown et al., 2020; Thoppilan et al., 2022; Chowdhery et al., 2023; Touvron et al., 2023). While these models are very effective for token generation, they incur a high inference cost as the model and its transient states need to be loaded into computing memory for each subsequently generated token. Moreover, scaling up these models, besides making each call more compute-intensive, also makes their autoregressive generation memory bound (Pope et al., 2023), preventing them from making effective use of available compute. This poses a significant challenge to the deployment of large autoregressive transformers, particularly for user-facing applications with tight latency requirements.

Given the memory-bound nature of large language model (LLM) inference, recent work (Leviathan et al., 2023; Chen et al., 2023) proposed Speculative Decoding as an effective technique to accelerate decoding based on concepts borrowed from speculative computation (Burton, 1985) to exploit the available extra compute. The core of speculative decoding is the idea of speculating multiple candidate future tokens first, and then verifying them all in parallel. To achieve this, as shown in Figure 1(a), a two-model paradigm approach is used: a small auxiliary "draft" model for candidate speculation and a large "target" model for verification (Leviathan et al., 2023; Chen et al., 2023). Although effective in accelerating LLMs, speculative decoding complicates deployment. Training also becomes more demanding and complicated, as a separate draft model needs to be trained and aligned with the target model. It is also not resource-efficient, requiring to host two models in memory during token prediction. This increased footprint is especially unsatisfactory for resource-constrained devices.

In this paper, we propose *Speculative Streaming*, a single-

[1]Apple. Correspondence to: Nikhil Bhendawade <nbhendawade@apple.com>, Irina Belousova <ibelousova@apple.com>, Qichen Fu <qfu22@apple.com>, Henry Mason <hmason@apple.com>, Mohammad Rastegari <mrastegari@apple.com>, Mahyar Najibi <najibi@apple.com>.

model speculative decoding approach that unifies speculation and verification, obviating the need for a separate draft model as shown in Figure 1(b). This is accomplished by incorporating multi-stream attention into the target model to perform n-gram prediction which serves as future candidate speculation. As a result, a forward model pass can verify the previously generated tokens while simultaneously speculating on the future tokens. Moreover, compared to previous approaches, Speculative Streaming is trained end-to-end, naturally aligning speculation and verification phases.

While making the system significantly simpler and resource efficient, Speculative Streaming achieves speedups comparable to two-stage speculative decoding (Leviathan et al., 2023) without degrading the quality metrics on a diverse set of downstream tasks. It also leads to on-par or better speedup than the more recent block-wise decoding model, Medusa (Cai et al., 2023), that introduces multiple additional high-dimensional prediction heads. Moreover, Speculative Streaming requires 10000X fewer additional parameters than Medusa(Cai et al., 2023), which makes it an ideal method for resource-constrained devices.

In summary, the advantages of Speculative Streaming are as follows:

- Achieving substantial speedups through streamlined fine-tuning and eliminating the need for a separate draft model.
- Demonstrating resource efficiency with 10000X fewer extra parameters compared to (Cai et al., 2023) while achieving better speedups, all without compromising quality across a diverse range of tasks.
- Simplifying deployment processes by eliminating the need to manage, align, and switch between two models during execution, as observed in (Leviathan et al., 2023).

2. Related Works

The inference of large language models is often limited by the sequential nature of auto-regressive decoding, where each token generation requires a complete network forward pass. Several approaches have been proposed to address the high inference latency of large language models by directly decreasing the memory footprint of LLMs. Model quantization (Frantar et al., 2022; Yao et al., 2022; Dettmers et al., 2023), knowledge distillation to a smaller a model (Gu et al., 2023; Agarwal et al., 2023), and pruning (Frantar & Alistarh, 2023; Sun et al., 2023a) are among these techniques. Recently, speculative decoding (SD) has emerged as a vital technique to accelerate autoregressive decoding.

The original speculative decoding approach (Chen et al., 2023; Leviathan et al., 2023) utilizes a smaller LLM (*a.k.a.* the *draft model*), to generate a candidate sequence of tokens to be verified by the *target model*. With a well-tuned draft model, this technique can achieve a 2-3x inference speedup. Recent SD variants propose parallel computation along the batch axis (Sun et al., 2023b), and tree-structured batches (Miao et al., 2023; Spector & Re, 2023) to improve the acceptance rates of the guessed tokens by the target model and to further boost the performance. However, these methods encounter a common limitation: the necessity of developing an accurate and independent draft model. First, training such a draft model aligned with the main model is not trivial (Zhou et al., 2023). Second, hosting two different models increases the system complexity, and is more computationally and operationally expensive to maintain.

Very recently, single-model speculation has also been considered. In particular, inspired by (Qi et al., 2020; Stern et al., 2018), Medusa (Cai et al., 2023) extends the main model to predict future tokens in parallel by training multiple output heads. While it does not require a draft model, each Medusa head of size (*hidden_size* $\times$ *vocab_size*) requires nonnegotiable additional parameters. As auto-regressive generation typically follows a memory-bound compute pattern, the significant amount of extra parameters introduces deployment challenges on resource-constrained devices. Alternatively, lookahead decoding (Fu et al., 2023) proposes a parallel decoding method without learning new parameters. It uses Jacobi Decoding and n-gram history trajectory cache to generate and verify future n-gram prediction. Differently, Speculative Streaming achieves n-gram generation by learning a set of token embeddings and accelerates decoding by running speculation and verification concurrently. Notably, our approach can achieve better speedup with significantly less number of extra parameters compared to the existing speculative decoding methods (Chen et al., 2023; Leviathan et al., 2023; Cai et al., 2023), while simplifying on-device deployment.

3. Method

3.1. Motivation

Most speculative decoding approaches exhibit a distinct separation in the training processes of draft and target models. However, directly using an off-the-shelf draft model often leads to sub-optimal performance in many downstream applications. The speculated tokens frequently fail the verification of the target model when draft and target models are misaligned. To achieve improved speedups, fine-tuning both draft and target models on downstream applications becomes necessary. Our objective is to devise an end-to-end trainable single-model framework capable of simultaneously predicting the next token and speculating future tokens. This eliminates the need for an auxiliary draft model while achieving speedups similar to those reported in (Leviathan et al., 2023). We aim to attain this speedup by increasing the arithmetic intensity of auto-regressive transformer calls without compromising generation quality.

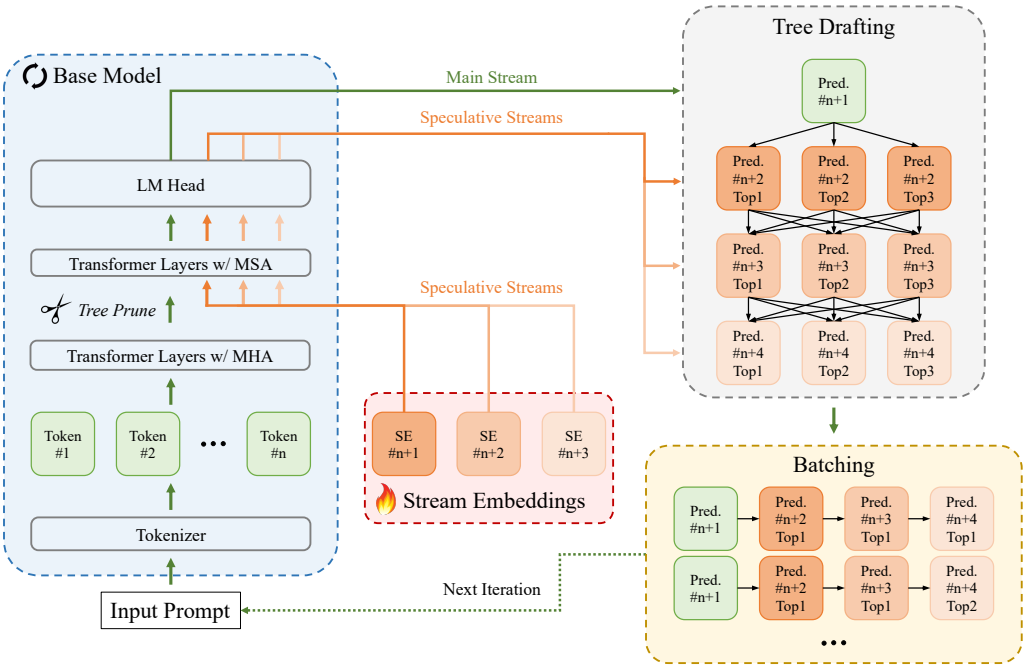


Figure 2. Architecture: We replace top N_s multi-head attention (MHA) layers of the base model with multi-stream attention (MSA) layers as described in (2). Speculative streams are initialized using hidden states of layer $N - N_s$ and stream identifier embeddings (SE), as described in (3) and used to generate speculative draft in the form of a tree. The speculative tree draft from the previous iteration is batched for verification and pruned before stream insertion. During each forward pass previous tree draft is verified and a new tree draft is issued using speculative streams as described in 3.2.2

3.2. Speculative Streaming

We propose Speculative Streaming to enable parameter efficient speculative fine-tuning and inference of decoder-only models on downstream applications. Compared to the standard draft-target speculative decoding (Leviathan et al., 2023) and more recently proposed block-wise decoding (Cai et al., 2023), Speculative Streaming introduces the following modifications. (a) Speculative stream design and initialization as described in Section 3.2.1 (c) Parallel speculation and verification as described in Section 3.2.2 (d) Parallel tree draft pruning, described in Section 3.2.3 and finally (e) Training objective as described in Section 3.2.4.

3.2.1. Streams Design and Initialization

Parameter efficient fine-tuning (Hu et al., 2022) of decoder-only pre-trained language models involves training low-rank adapters to predict next target token y_t given context tokens $(x_1....x_m)$ and previous target tokens $(y_1..y_{<t})$ on downstream applications. To inherently embed a notion of future token planning, we modify the training objective of the target model from next token prediction to n-gram prediction using the concept of multi-stream attention (Qi et al., 2020; Yang et al., 2019). This objective allows the model to plan for future tokens and prevent over-fitting on local correlations. In addition, each of the γ streams generates speculative tokens with negligible latency overhead when the model is memory-bound. Specifically, each added stream predicts $p(y_{t+j}|y_{<t}, x)$, where $1 <= j <= \gamma$, while main stream predicts $p(y_t|y_{<t}, x)$. We refer to the multi-head attention mechanism depicted in (Vaswani et al., 2017) as main-stream self-attention and introduce γ additional self-attention streams to speculate future tokens.

The attention mechanism of main stream is the same as standard multi-head attention (Vaswani et al., 2017).

$$M_t^{k+1} = \text{MHA}(M_t^k, M_{\leq t}^k, M_{\leq t}^k) \tag{1}$$

Where M_t^k denotes hidden states of main stream at layer k and time step t and $MHA(H, H, H)$ denotes attention between query HW^Q, key HW^K and value HW^V as described in (Vaswani et al., 2017). On the other hand, each speculative stream j at time step t attends to previous main stream hidden states as well as speculative stream hidden states as:

$$S_{tj}^{k+1} = \text{MHA}(S_{tj}^k, M_{\leq t}^k \oplus S_{t(\leq j)}^k, M_{\leq t}^k \oplus S_{t(\leq j)}^k) \tag{2}$$

where M_t^{k+1} and S_t^{k+1} refer to main and speculative

streams at time step t and layer k. Hidden state of last transformer layer N, M_t^N is used to predict y_t, whereas each speculative stream at last layer, S_{tj}^N predicts y_{t+j}. We refer to layers incorporating the attention mechanism in Equation (1) as MHA layers while layers incorporating speculative stream attention Equation (2) are referred to as MSA layers.

Key/value projections of main stream hidden states are cached during inference to avoid re-computation, whereas, speculative stream attention is specifically designed to avoid storing additional key/value projections associated with individual streams. This is because speculative streams are trained to learn contextual features from main stream key/value context allowing us to not introduce additional caching overhead and operate within memory bounds of resource-constrained devices during inference. We initialize hidden states of speculative streams at layer $N - N_s$ instead of initializing them from the embedding layer, where $N_s < N$. Specifically, stream j at time t is initialized at layer $N - N_s$ as,

$$S_{tj}^{N-N_s} = f_\eta(M_t^{N-Ns}) + P_j^{N-N_s} \tag{3}$$

where P_j is a stream identifier embedding that embeds a sense of relative position into streams and distinguishes the computation from main stream. f_η is a linear transformation of rank η to transform main stream hidden states into speculative stream hidden states. This initialization helps to reduce computation per forward pass, since only the main stream needs to be passed through $N - N_s$ layers, while speculative streams are passed through the last N_s layers, decreasing the speculative FLOPs contribution by $(N - N_s)/N$ and in turn helping with peak power consumption on the device. In terms of forward pass latency, FLOPs do not contribute significantly when the model is memory bound, however, as we describe in Section 3.2.2, we sample additional tokens to make the model compute-bound, therefore FLOP reduction becomes crucial. Initialization with a hidden state of middle-upper transformer layers may also help with the future token prediction as $M^{(N-Ns)}$ itself contains high-level contextual features to aid with the prediction of future n-grams (Pal et al., 2023). We also experimented with value rotation based stream design which does not require identifier embeddings and incurs no parameter overhead as described in B.

3.2.2. Parallel speculation and verification

In standard draft-target speculative decoding (Leviathan et al., 2023), speculation and verification processes happen sequentially. The draft model waits for the target model to issue a correction before generating the next draft. The target model also needs to wait for the speculative draft to be generated. Speculative Streaming makes this process more efficient by parallelizing speculation and verification. In each forward pass, the draft generated in the previous step is verified and a new draft is generated as shown in Figure 2. For instance, in step s, if draft tokens $(\tilde{y_1}..\tilde{y_\delta})$ are accepted where $0 < \delta \leq \gamma$, main stream M_δ is used to issue a correction token and logits from speculative streams $S_{\delta(1...\gamma)}$ are used to generate draft for step $s + 1$.

Instead of using a linear sequence of speculated tokens for verification, we sample a tree of tokens from main and speculative streams, such that each path in the tree is one possible verification candidate. Tree drafting enables accepting the longest matching candidate sequence and more tokens can be advanced during each forward pass. To create a tree draft, instead of sampling 1 token from logits of speculative streams, $(z_1...z_\gamma)$, we sample top k tokens and form a tree of sampled tokens as shown in Figure 2, such that tokens sampled from stream n are predecessors of tokens sampled from stream $n + 1$. We process a tree draft of speculative tokens in one forward pass by creating an additive attention mask (Vaswani et al., 2017) such that each node in the tree attends to its predecessor. Attention mask between k^{th} token sampled from logits of stream j, $\tilde{y}_{jk}$ and the mth token sampled from logits of stream n, $\tilde{y}_{nm}$ is

$$a_{\tilde{y}_{jk}\tilde{y}_{nm}} = \begin{cases} 0 & \text{if j = n+1,} \\ -\infty & \text{otherwise} \end{cases} \tag{4}$$

Please refer to Figure 8 for more details. It's worth noting that for a fixed γ and k, the attention mask remains constant in each forward pass and enables effective batching.

3.2.3. Parallel tree pruning

One of the issues with the naive creation of a speculative tree draft is that every permutation between k tokens sampled from each stream needs to be considered as a viable speculative candidate for the next verification pass. For instance, sampling k tokens from each of γ streams results in tree draft of size 1 + $\sum_{g=1}^{\gamma} k^g$. Furthermore, each of the draft tokens is batched with γ speculative streams in MSA layers to ensure that the generation of the next draft happens in the same forward pass, resulting in a batch size of $(1 + \gamma) * (1 + \sum_{g=1}^{\gamma} k^g)$. As batch size increases, target model inference becomes compute-bound, obviating the latency benefit of sampling more tokens. We mitigate this problem by introducing a parallel tree draft pruning layer, which prunes some of the tokens from the input tree draft based on transition probability between parent and immediate child tokens. To obtain transition probabilities without using proxy models, we use an early-exiting-based technique. Specifically, hidden states of the main stream at layer l, M^l are passed through a low-rank linear transformation o_θ, where the rank θ is typically set to a small value like 8 to keep parameter overhead small. We use

original language modeling head, H to obtain early exit logits, $\tilde{z} = H(o_\theta(M^l)$. $\tilde{z}_{pc}$ is used to approximate transition probability between parent token p and child token c. The pruning layer can be inserted at any point in the network, guided by the trade-off between forward pass latency and pruning accuracy. Early insertion reduces latency but risks pruning potentially valuable tokens. Conversely, late insertion retains more "good" tokens but comes at the cost of increased forward pass latency. In all experiments described in Section 4.1, we insert the pruning layer just before speculative stream insertion empirically. More details can be found in Figure 7.

3.2.4. Training objective

To efficiently generate future n-grams, we finetune the base model jointly on the prediction loss of the next token as well as γ future tokens as

$$L_{ss} = -\alpha_0 \left(\sum_{t=1}^{T} \log p_\theta(y_t|y_{<t}, x)\right) - \sum_{j=1}^{\gamma} \alpha_j \left(\sum_{t=1}^{T-j} \log p_\theta(y_{t+j}|y_{<t}, x)\right) \quad (5)$$

where α_0 and α_j are set empirically to normalize losses of the next token and speculative tokens prediction. Tree-pruning adapter described in Section 3.2.3 can be trained on the next token prediction loss, either jointly along with main and speculative streams or post-training of streams. Training times vary based on the number of MSA layers but are comparable to (Cai et al., 2023) style approach for $N_s = 4$. For experiments described in 4, our recipe involves training LoRA adapters for 5 epochs on the downstream datasets in BFloat16, using the AdamQ optimizer, a learning rate of 5e-4, and a linear scheduler. For tree pruning (see Section 3.2.3), we use a low-rank linear transformation of rank 8 to keep parameter overhead minimal.

4. Experiments

We evaluate our methods on the pre-trained models of various scales and a diverse set of downstream applications.

Dataset. We test our methods on a diverse set of applications that are vital to on-device AI assistants, namely Structured Queries, Text Summarization, and Meaning Representation. We specifically choose fine-tuning setup since it has been a norm to share a base model and use application-specific adapters for user-facing applications. We use the Dialogsum (Chen et al., 2021) dataset for Text Summarization, the sql-create-context dataset built from WikiSQL (Zhong et al., 2017) and SPIDER (Yu et al., 2018) for Structured Queries, and e2e-nlg dataset (Dušek et al., 2020) for Meaning Representation.

Model Configuration. We tested four different open source models of various scales, Phi(1.3B)(Li et al., 2023), Open-llama(7B)(Touvron et al., 2023), and OPT(1.3B, 6.7B) (Zhang et al., 2022). We compare our method with the standard draft-target speculative decoding ((Leviathan et al., 2023)) and single-model speculative decoding framework, Medusa (Cai et al., 2023). For the standard draft-target approach, we use OPT-125m, the smallest configuration of available open-source OPT models as the draft model.

Baselines To compare with Medusa (Cai et al., 2023) style approach, we use pre-trained base models with LoRA adapters (Hu et al., 2022) of rank 32 and Medusa heads as the baseline, and Speculative Streaming with the same base models, stream embeddings and LoRA adapters as target. Medusa heads are trained following the recipe described in (Cai et al., 2023). Both Medusa heads and the number of maximum streams are fixed to 4 and the residual blocks per head used in Medusa are set to 1. For comparison with standard draft-target speculative decoding (Leviathan et al., 2023), we use OPT models since they come with different configurations and sizes. OPT-125m is deployed as a draft model while OPT-1.3b and OPT-6.7b are used as target models since a ratio of 10-100X is typically considered to be optimal. Note that, similar to the target model, only LoRA adapters of the draft model are fine-tuned on downstream applications because fine-tuning the entire draft model on each downstream application is not practical in on-device settings. Also, LoRA fine-tuning tends to achieve on-par performance to full-model fine-tuning (Hu et al., 2022).

4.1. Results

4.1.1. Overview

We report wall-time speedups and generation quality metrics on test split using a batch size of 1 on a single Nvidia A100-80G GPU. Inference is performed in float16 using greedy sampling and $T = 0$. Please refer to Appendix A.2 for more experimental details and Appendix B for ablations on top-k sampling and $T = 1$. We use Exact Match (EM) accuracy metric for the structured query task and Rouge1/RougeLSum metrics for the Dialog Summarization and Meaning Representation tasks. We use N_s/N of $1/6$ for the structured query task and $1/2$ for the summarization and meaning representation task. N_s is chosen to ensure the generation metric is on-par with the baseline. Details on the effect of N_s on generation metric are found in Section 4.2.

Table 1 presents the comparison between standard auto-regressive decoding baseline, Medusa, and our approach in terms of speedup, call reduction ratios, and the number of extra parameters. We find that across a variety of downstream tasks, the walltime speedups and call reduction ratios of Speculative Streaming are consistently on-par/higher than Medusa while incurring significantly lesser parameter

Table 1. Walltime speedup, CR ratio, parameter overhead, and Metric comparison using different models fine-tuned on downstream applications. CR ratio denotes acceleration agnostic target model call reduction ratio. We use exact match accuracy as a metric for SqlContext, and Rouge1/RougeLSum as a metric for Dialogsum and E2E-NLG tasks.

Dataset	Model	Method	SpeedUp (↑)	CR Ratio (↑)	Metric (↑)	# Extra Parameters (↓)
SqlContext	OPT-1.3b	Baseline	1.00	1.00	84.98	–
		Medusa	2.07	2.79	84.98	$4.28E8$
		SS (ours)	**2.39**	**3.57**	**87.40**	$4.096E4$
	PHI-1.3b	Baseline	1.00	1.00	88.71	–
		Medusa	2.58	3.25	88.71	$4.36E8$
		SS (ours)	**2.62**	**3.53**	**89.90**	$4.096E4$
	OpenLlama-7b	Baseline	1.00	1.00	89.88	–
		Medusa	**3.20**	4.10	90.11	$5.91E8$
		SS (ours)	3.14	**4.13**	**91.70**	$8.19E4$
DialogSum	OPT-1.3b	Baseline	1.00	1.00	43.40/35.56	–
		Medusa	1.56	1.91	43.40/35.50	$4.28E8$
		SS (ours)	**1.94**	**2.62**	**44.07/35.99**	$4.096E4$
	PHI-1.3b	Baseline	1.00	1.00	**43.57/35.60**	–
		Medusa	**1.89**	2.28	**43.57/35.60**	$4.36E8$
		SS (ours)	1.83	**2.34**	43.36/35.31	$4.096E4$
	OpenLlama-7b	Baseline	1.00	1.00	**44.20/36.50**	–
		Medusa	1.76	2.25	**44.20/36.50**	$5.91E8$
		SS (ours)	**1.87**	2.51	43.92/35.70	$8.19E4$
E2E-NLG	OPT-1.3b	Baseline	1.00	1.00	**69.48**/50.17	–
		Medusa	2.13	2.95	**69.48**/50.17	$4.28E8$
		SS (ours)	**2.45**	**3.72**	69.32/**50.51**	$4.096E4$
	PHI-1.3b	Baseline	1.00	1.00	**67.90**/48.50	–
		Medusa	2.78	3.35	**67.90**/48.50	$4.36E8$
		SS (ours)	**2.84**	**3.69**	67.40/**48.52**	$4.096E4$
	OpenLlama-7b	Baseline	1.00	1.00	**69.50/50.30**	–
		Medusa	2.70	3.22	**69.50/50.30**	$5.91E8$
		SS (ours)	**2.96**	**3.55**	68.66/49.56	$8.19E4$

overhead. Furthermore, as summarized in Table 2, our approach achieves better wall-time latencies than the standard draft-target speculative decoding since the difference in the number of target calls between both approaches is not large enough to offset auto-regressive drafting overhead. All wall-time latencies are reported using open-source versions of models available on (Wolf et al., 2019) and it is possible that further optimizing draft and target models using efficient inference techniques (Nvidia, 2024) or quantization (int4/8) may lead to lower latencies. Finally, It's worth noting that the generation metrics of our method are consistently comparable with LoRA fine-tuned base models making it an excellent alternative to next-token prediction-based fine-tuning.

4.1.2. Analysis and Insights

Without Auxiliary Models Medusa heads generate each token independently from the shared hidden state of the last layer, and dependency between speculative tokens predicted by medusa heads, $y_{(t+1..t+\gamma)}$ and next token y_t predicted by the base model at time step t may not be well captured since there no attention mechanism involved. On the other hand, speculative streams attend to the main stream and each other, capturing the token dependency, resulting in better call reduction ratios than Medusa. In terms of parameters, each Medusa head adds about $h^2 + hv$ parameters, where h is the hidden size and v is the vocabulary size. The number of Medusa heads also scales linearly w.r.t. γ, the length of the speculative window, which in turn increases parameter overhead linearly with γ. On the other hand, Speculative Streaming uses speculative adapters which do not scale with γ. Although, Stream identifier embeddings scale with γ, the parameter overhead associated with each embedding is linear to h. Furthermore, in fine-tuning settings "speculative adapter" parameters are shared with base model adapters, therefore, parameter overhead associated with our approach is just γh.

With Auxiliary Models Speculative Streaming consistently

Table 2. Walltime latency (per sample) comparison with standard draft-target based speculative decoding approach using OPT-125m as the draft model for $\gamma = 4$. Although calls to target model using our approach are higher than draft-model-based speculative decoding, it does not incur auto-regressive drafting overhead, achieving better latency on OPT-1.3b and OPT-6.7b models. We use exact match accuracy as a metric for SqlContext, while Rouge1/RougeLSum is used as a metric for Dialogsum and E2E-NLG tasks.

Dataset	Target	Method	Target calls	Draft Calls	Walltime Latency (ms, ↓)	Metric (↑)
SqlContext	OPT-1.3b	Two-model SD	6.59	22.35	269.24	84.98
		SS (ours)	7.79	0	**133.48**	**87.40**
	OPT-6.7b	Two-model SD	6.60	22.41	301.10	89.13
		SS (ours)	6.88	0	**157.04**	**89.34**
Dialogsum	OPT-1.3b	Two-model SD	11.65	42.59	493.59	43.40/35.60
		SS (ours)	13.41	0	**248.26**	**44.07/35.99**
	OPT-6.7b	Two-model SD	12.15	35.76	555.99	**44.40/36.60**
		SS (ours)	14.39	0	**442.83**	44.30/36.30
E2E-NLG	OPT-1.3b	Two-model SD	8.86	31.47	345.72	**69.48**/50.17
		SS (ours)	9.80	0	**164.23**	69.32/**50.51**
	OPT-6.7b	Two-model SD	8.90	31.58	412.02	**69.34/49.88**
		SS (ours)	10.26	0	**243.62**	69.07/49.69

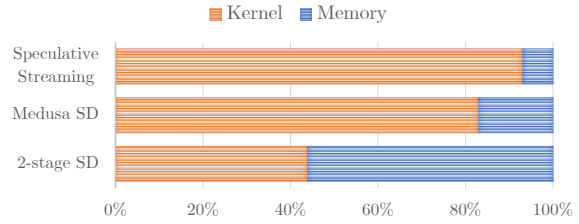


Figure 3. Speculative Streaming speeds up decoding by increasing arithmetic intensity of memory bound auto-regressive decoding step. Kernel and memory utilization of OPT-1.3b model with Medusa-style approach and draft model (OPT-125m) based speculative decoding approach is also shown for comparison.

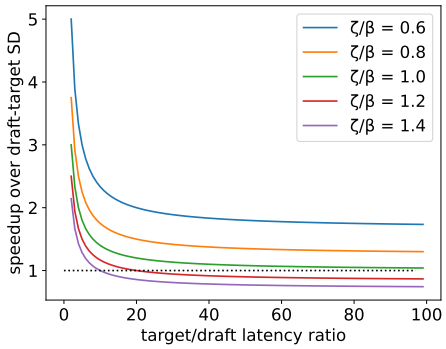


Figure 4. Speculative Streaming speedup over draft-based speculative decoding for different ζ/β and target/draft latency ratios, where ζ denotes the number of advancements per verification step for draft-based speculative decoding while β denotes the same for Speculative Streaming.

achieves lower wall time latency than standard draft-target speculative decoding as depicted in Table 2. It's worth noting that, target model calls of draft-target speculative decoding are lower than Speculative Streaming, however, it comes at the cost of auto-regressively running draft model γ times to generate speculative draft. On the other hand, draft generation with Speculative Streaming incurs almost no additional latency overhead, as target model decoding tends to be memory-bound even with increased tree draft size. This translates to increased kernel utilization and arithmetic intensity as shown in Figure 3. Draft-based approach on the other hand has low kernel utilization because of the memory-bound nature of auto-regressive drafting.

An argument could be made that a smaller draft model may perform better since drafting should cost less, but acceptance rates may drop as well as the draft model size is decreased. To formalize the comparison with standard draft-target speculative decoding, we do the following analysis, let's say, C_{draft} is the latency cost associated with forward pass through the draft model, C_{target} is the cost associated with forward pass through target model, while C_{ss} is cost associated with speculative streaming forward pass. ζ is the number of decoding tokens advanced during the verification step for the draft-target approach while β is the number of tokens advanced in Speculative Streaming. We equate latency cost associated with single token advancement to compare both approaches.

$$(\gamma * C_{draft} + C_{target})/\zeta = C_{ss}/\beta \quad (6)$$
$$(\gamma + C_{target}/C_{draft})/\zeta = (C_{ss}/C_{draft})/\beta$$

Assuming $\gamma = 4, C_{target}/C_{draft} = 10$, and $C_{ss} \approx C_{target}$, $\zeta = 1.4\beta$, meaning that advancements per veri-

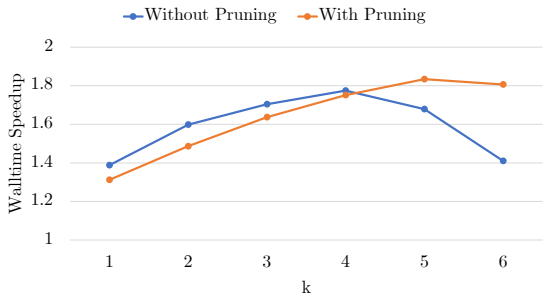


Figure 5. As more tokens (k) are sampled from each stream keeping γ fixed for the creation of a tree draft, walltime speedup increases due to the increased number of candidates. This trend reverses as k continues to increase and the model transits into the compute-bound phase. Pruning less probable paths from tree draft helps to reduce compute for higher values of k thereby reducing latency per forward pass and offering more speedup.

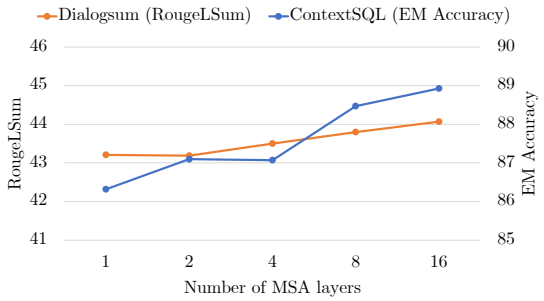


Figure 6. As the number of multi-stream attention layers increases, metrics on downstream tasks improve as well. We use RougeLSum as the metric for the Dialogsum task, and Exact Match (EM) accuracy as the metric for the ContextSQL task.

fication step in standard draft-target approach have to be 1.4X of Speculative Streaming to achieve wall time latency parity. Note that, this analysis ignores cache adjustment overhead and prompt processing overhead, but provides valuable intuition to guide the choice between draft-target vs Speculative Streaming approaches. We also analyze under which settings speculative streaming is likely to offer more benefits as compared to the standard draft-target approach. Fig. 4 shows theoretical speedups of Speculative Streaming over draft-target based approach for different Target to draft latency ratios. As the latency ratio increases, the draft-target approach is likely to offer more speedup benefits when $\zeta/\beta > 1$, meaning that when the draft model is accurate enough to achieve more token advancements per target model verification step than Speculative Streaming and also small enough to yield higher latency ratios, it is likely to benefit more. Finding/creating such a model usually requires significant engineering efforts. In downstream application settings, finding ideal draft models becomes even more challenging since ζ tends to vary based on application. If applications share the draft model and only train adapters, the draft model may not remain small enough to meet target-to-draft latency ratios, making it challenging to achieve more speedups than Speculative Streaming.

4.2. Ablations

Speculative Draft Size. To improve the acceptance rate of the tree draft, we try various settings of γ, the number of speculative positions, and k, the number of sampled tokens per speculative position. Figure 5 shows walltime speedup for $\gamma = 3$. As we sample more tokens from each speculative position, advancement per forward pass, β increases since more candidates are available for verification, leading to more speedup. However, as we continue to increase k, forward pass latency overhead becomes more prevalent as the model transitions into compute-bound phase and the speedup reverses the course. This is because naively forming a tree draft leads to an exponential increase in batch size with k as described in 3.2.3. We insert a tree pruning layer to remove less probable paths and reduce the size of the tree draft. Pruning tree draft reduces forward pass latency, and a well calibrated threshold ensures that only noisy paths in the tree get pruned. Tree pruning tends to help with walltime speedup as k continues to increase as shown in Figure 5.

Number of MSA Layers There are trade-offs involved in deciding the number of MSA layers to incorporate in terms of downstream generation metric, training time, and FLOPs increase. As we increase the number of MSA layers, the generation metric improves and this trend remains the same across different downstream tasks. Typically incorporating MSA in the top 2 - 8 layers offers a good trade-off between metric, FLOPs increase and training time. Figure 6 shows the generation performance of the OPT-1.3b model on Structured Query and Summarization tasks.

5. Conclusion

In this paper, we proposed Speculative Streaming, a method to accelerate decoding of large language models. Compared to the standard speculative decoding approaches, Speculative Streaming removes the need for an auxiliary "draft" model. Instead, it unifies speculation and verification by efficiently fusing multiple speculative streams into a single "target" model. Speculative Streaming simplifies the fine-tuning process and achieves on-par or better speed-up and quality compared to previous approaches. It is also parameter efficient and removes the need for loading two models into the memory, making it a suitable approach for resource-constrained scenarios.

Acknowledgements

We would like to thank Sachin Mehta, Moin Nabi, Antonie Lin, Minsik Cho, Arsalan Farooq, and Jason Williams for their valuable feedback and discussions.

References

Agarwal, R., Vieillard, N., Stanczyk, P., Ramos, S., Geist, M., and Bachem, O. Gkd: Generalized knowledge distillation for auto-regressive sequence models. *arXiv preprint arXiv:2306.13649*, 2023.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Burton, F. W. Speculative computation, parallelism, and functional programming. *IEEE Transactions on Computers*, 100(12):1190–1193, 1985.

Cai, T., Li, Y., Geng, Z., Peng, H., and Dao, T. Medusa: Simple framework for accelerating llm generation with multiple decoding heads. https://github.com/FasterDecoding/Medusa, 2023.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Chen, Y., Liu, Y., Chen, L., and Zhang, Y. DialogSum: A real-life scenario dialogue summarization dataset. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 5062–5074, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-acl.449. URL https://aclanthology.org/2021.findings-acl.449.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.

Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefler, T., and Alistarh, D. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*, 2023.

Dušek, O., Novikova, J., and Rieser, V. Evaluating the State-of-the-Art of End-to-End Natural Language Generation: The E2E NLG Challenge. *Computer Speech & Language*, 59:123–156, January 2020. doi: 10.1016/j.csl.2019.06.009.

Frantar, E. and Alistarh, D. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International Conference on Machine Learning*, pp. 10323–10337. PMLR, 2023.

Frantar, E., Ashkboos, S., Hoefler, T., and Alistarh, D. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Breaking the sequential dependency of llm inference using lookahead decoding, November 2023. URL https://lmsys.org/blog/2023-11-21-lookahead-decoding/.

Gu, Y., Dong, L., Wei, F., and Huang, M. Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*, 2023.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pp. 19274–19286. PMLR, 2023.

Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., and Lee, Y. T. Textbooks are all you need ii: **phi-1.5** technical report. *arXiv preprint arXiv:2309.05463*, 2023.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Wong, R. Y. Y., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. Specinfer: Accelerating generative llm serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 2023.

Nvidia. Fastertransformer, 2024. URL https://github.com/NVIDIA/FasterTransformer.

Pal, K., Sun, J., Yuan, A., Wallace, B. C., and Bau, D. Future lens: Anticipating subsequent tokens from a single hidden state. *arXiv preprint arXiv:2311.04897*, 2023.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5, 2023.

Qi, W., Yan, Y., Gong, Y., Liu, D., Duan, N., Chen, J., Zhang, R., and Zhou, M. Prophetnet: Predicting future n-gram for sequence-to-sequence pre-training. *arXiv preprint arXiv:2001.04063*, 2020.

Spector, B. and Re, C. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*, 2023.

Stern, M., Shazeer, N., and Uszkoreit, J. Blockwise parallel decoding for deep autoregressive models. *Advances in Neural Information Processing Systems*, 31, 2018.

Sun, M., Liu, Z., Bair, A., and Kolter, J. Z. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*, 2023a.

Sun, Z., Suresh, A. T., Ro, J. H., Beirami, A., Jain, H., and Yu, F. Spectr: Fast speculative decoding via optimal transport. *arXiv preprint arXiv:2310.15141*, 2023b.

Thoppilan, R., De Freitas, D., Hall, J., Shazeer, N., Kulshreshtha, A., Cheng, H.-T., Jin, A., Bos, T., Baker, L., Du, Y., et al. Lamda: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019.

Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R. R., and Le, Q. V. Xlnet: Generalized autoregressive pretraining for language understanding. *Advances in neural information processing systems*, 32, 2019.

Yao, Z., Yazdani Aminabadi, R., Zhang, M., Wu, X., Li, C., and He, Y. Zeroquant: Efficient and affordable post-training quantization for large-scale transformers. *Advances in Neural Information Processing Systems*, 35: 27168–27183, 2022.

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. *arXiv preprint arXiv:1809.08887*, 2018.

Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

Zhong, V., Xiong, C., and Socher, R. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017.

Zhou, Y., Lyu, K., Rawat, A. S., Menon, A. K., Rostamizadeh, A., Kumar, S., Kagy, J.-F., and Agarwal, R. Distillspec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*, 2023.

A. Implementation Details

A.1. Tree Draft Management

In this section, we go into more detail about tree draft sampling, flattening, and pruning. As shown in the main paper, when processing prompt $(x_1...x_t)$, we insert speculative streams along with the last token to generate logits, z_t corresponding to main stream and $(z_{t1}...z_{t\gamma})$ corresponding to speculative streams. Tree draft is sampled following the procedure described in Section 3.2.2. The sampled draft is then flattened along the sequence length dimension and the attention mask is composed such that child nodes attend to their predecessors starting with root as shown in Figure 7 and Figure 8. The root token of the tree draft is the correction issued by main stream. Each iteration after prompt processing involves verifying the previous tree draft and sampling a new one. After passing the tree draft through $N - N_s$ layers, we use contextual features learned by middle layers to approximate transition probability between parent and child tokens. As shown in Figure 7, since the transition probability between token "$parameter''$ and "$compare''$ is less than a set threshold, we prune the sub-tree starting from "$compare$" in the feature domain , and m_2, m_5, m_6 are pruned. Please note that the key value cache of layers $0..(N - N_s - 1)$ before the pruning layer is not trimmed at this point to keep pruning latency overhead minimal. Key value cache backtracking is done lazily after each generation step. Speculative streams are inserted alongside each node in the pruned draft. Layers $(N - N_s..N)$ use Multi-stream attention as described in Equation (2). The verification procedure finds the longest matching path in the pruned tree that main stream can accept. As shown in Figure 7, path ("$parameter''$, "$efficient''$, "$speculative''$) is accepted. Correction token sampled from logits of main stream corresponding to last accepted token, m_1 becomes new root while tokens sampled from logits of streams (s_{10}, s_{11}) form the sub-tree.

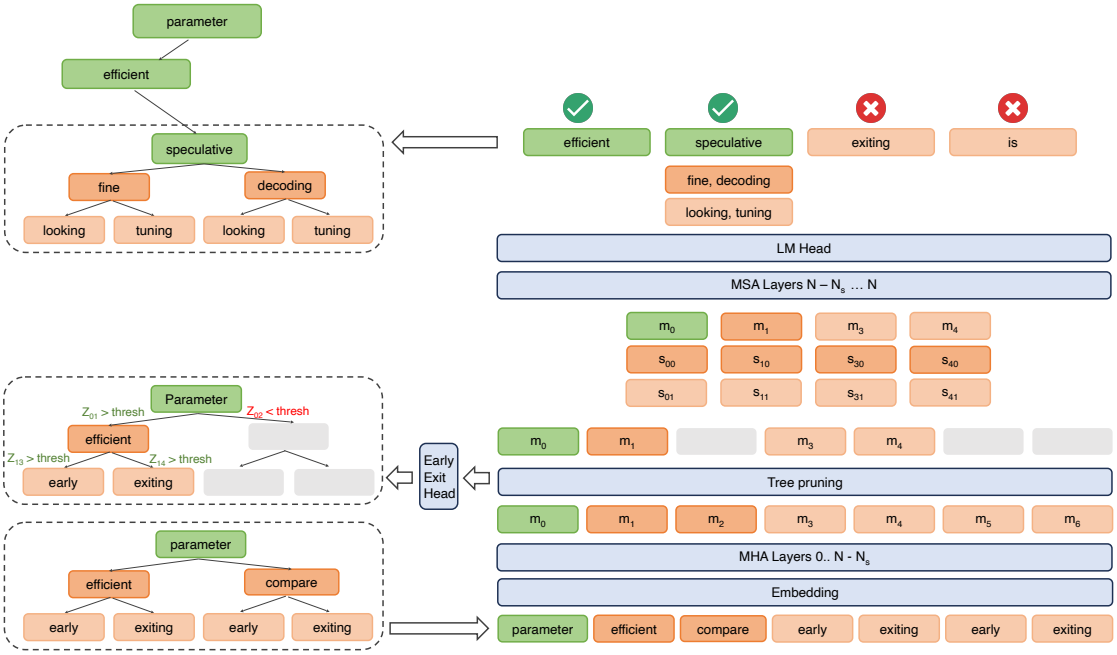


Figure 7. Parallel tree draft speculation and verification: Tree draft from the previous iteration is flattened for verification. After $N - N_s$ MHA layers, the tree pruning procedure obviates less probable tokens based on transition probability between parent and child tokens. In this illustration Z_i denotes normalized early exit logits corresponding to main stream at index i, m_i, while Z_{ij} denotes transition probability between token at index i and j in flattened tree draft. The verification procedure is subsequently run on the pruned tree and speculative tokens are sampled from streams corresponding to the latest accepted token. In above illustration, "$speculative''$, "$fine, decoding''$ and "$looking, tuning''$ are sampled from streams m_1, s_{10} and s_{11}.

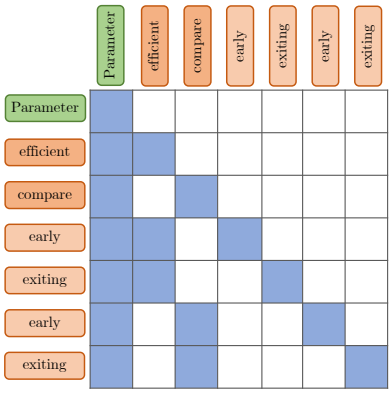
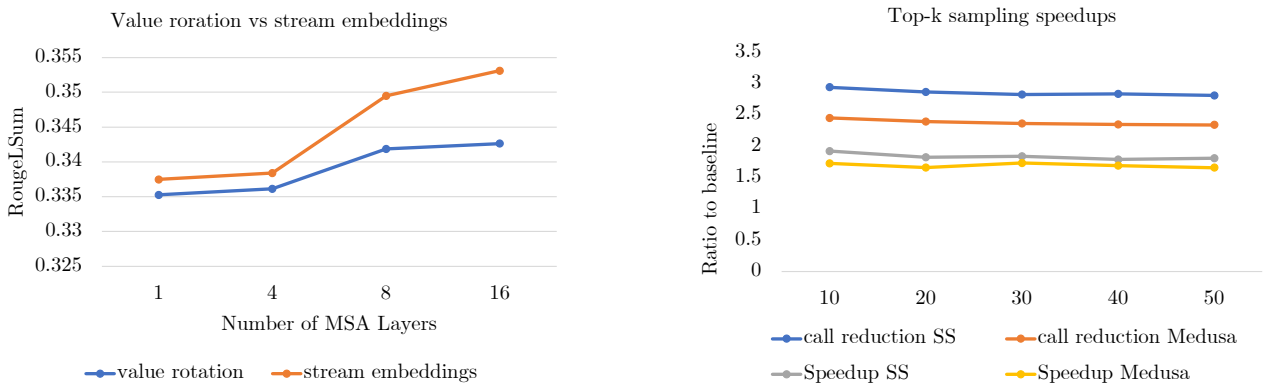


Figure 8. Attention mask for tree draft is composed in such a way that child tokens can attend to all predecessors starting from root, root being correction issued by main stream. In this illustration, "$early$" attends to "$parameter$" and "$efficient$" and itself since "$parameter - efficient - early$" forms one path in tree. "$early$" is also replicated to form another path "$parameter - compare - early$". This attention mask allows batching multiple paths and increasing acceptance rate as number of candidates increase.

A.2. Experimental Setup Details

Fine-tuning procedure for both baseline and target approaches described in the paper in Section 4 involves training LoRa adapters for 5 epochs. We set $\alpha_0 = 1$ and $\alpha_j = 0.1$ for $j = 1...\gamma$ to weigh speculative loss relative to next token prediction loss for both baseline and target methods. We experimented with linear transformations of different ranks to initialize speculative streams from main stream as described in Equation (3), however we find that simply using identity transformation achieves similar performance with much less parameter overhead. We use identity transformation for all the experiments described in Section 4. We report best results for Medusa and our approach over different γ and k values, while for standard draft-target speculative decoding approach k is fixed to 1. We also report accelerator agnostic speedups (call reduction ratios) assuming negligible verification and draft composition overhead as latency of forward pass, verification and draft composition procedures vary greatly depending on accelerator (*e.g.* a mobile device neural engine *vs.* Nvidia A100), while call reduction ratio metric tends to serve as roof-line for achievable speedup. Lastly, we use "hard" matching criteria for verification of speculative draft. Relaxing this criteria to "soft" matching may yield higher speedups (Cai et al., 2023)



(a) Value Rotation

(b) Top-k Sampling

Figure 9. (a) We analyze the effect of value projection rotation on RougeLSum scores of the Dialog summarization task using PHI-1.3b as the base model for different numbers of MSA layers. Each stream is rotated in proportion to the distance from the main stream. (b) We study the effect of top-k sampling on wall-time speedups and call reduction ratios for Speculative Streaming (SS) and Medusa style approaches using OPT-1.3b as a base model on the Meaning Representation task.

B. Ablation:

Value Rotation We analyzed more ways of differing computation of main stream from speculative streams. Apart from using dedicated stream embeddings, one way to differentiate the computation while incorporating a sense of relative position is simply rotating streams relative to each other. In this ablation, we initialize each stream with the main stream hidden state and rotate the value projection during attention computation in the proportion of the relative distance from main stream as :

$$V_{tn}^k = V_t^k e^{i\epsilon n} \tag{7}$$

Where $1 <= n <= \gamma$ is stream index, V_t^k denotes value projection of main stream at time step t and layer k, while V_{tn}^k denotes value projection of stream n, $0 \leq \epsilon \leq \frac{\pi}{2N}$ denotes an arbitrary rotation step and N denotes the sum of maximum sequence length and number of streams. Figure 9 (a) shows the effect of using value rotation on Rouge scores on the Dialog Summarization task with the Phi-1.3b model. Downstream metric for value rotation-based approach tends to be lower than using dedicated stream embeddings across different settings of MSA layers, however, the trend of increasing metric with added MSA layers remains the same. It's worth noting that for $N_s = 16$, simply rotating value projections achieve better metrics than using $N_s = 4$ with dedicated stream embeddings.

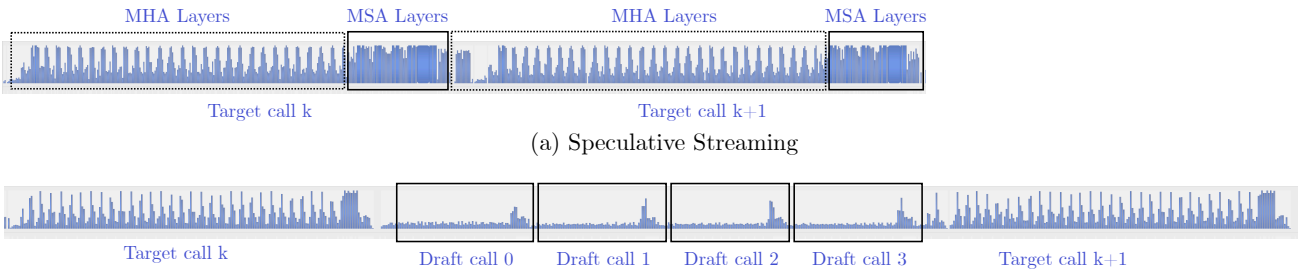
Top-k Sampling In the main paper, we reported speedup results using greedy sampling and T=0. To further analyze speedups in the Top-k sampling regime, we try various values of k and T = 1 for both Medusa style and Speculative Streaming approaches. Figure 9 (b) shows the effect of increasing k on the walltime speedups and call reduction ratios. Although increasing k leads to lower wall-time speedups for both baseline and target methods due to stochastic rejection of tokens, our approach retains its lead achieving better call reduction ratios and walltime speedups across different values of k.

C. Compute and Memory Profiling

The draft overhead associated with the standard draft-target speculative decoding approach tends to be non-trivial especially when the latency ratio between target and draft models $c_{target}/c_{draft} <= 10$. This is because speculation and verification procedures are run in serial manner. Figure 10 shows the kernel utilization timeline when OPT-125m is used as a draft while OPT-1.3b model is used as the target. Auto-regressive draft generation decreases overall kernel utilization in draft-target approach, while additional computation involved in MSA layers increase kernel utilization in case of Speculative Streaming thereby efficiently utilizing the accelerator and speeding up the decoding process. Negligible cost draft models may offer a better choice to keep kernel utilization at higher levels in case of draft-target approach, however, acceptance rates tend to drop as draft model size decreases.

D. Qualitative Examples

In this section, we present qualitative examples to illustrate the effectiveness of Speculative Streaming. By examining specific instances, we aim to highlight how this approach enhances the overall performance of the decoding process. An example of the SQL query generation task is shown in Figure 11, while a dialog summarization example is shown in Figure 12. Each row indicates the previous sequence of accepted draft tokens (in black) and the new sequence of generated tokens in green/red. We use $\gamma = 4$ and $k = 1$ to illustrate the decoding process. Green tokens in each row indicate tokens accepted in the next forward pass, while red tokens indicate tokens rejected in the next forward pass. Speculative Streaming appears to generate meaningful drafts with high acceptance rates by capturing dependencies between tokens quite effectively, despite generating them in a non-auto-regressive manner.



(a) Speculative Streaming

(b) Two Stage Speculative Decoding

Figure 10. Kernel utilization timeline for speculative streaming and the standard draft-target speculative decoding. Draft-target approach runs speculation and verification in serial manner while it is parallelized in Speculative Streaming. Auto-regressive draft generation often has low kernel utilization as shown leading to decreased overall kernel utilization while MSA layers in Speculative Streaming increase kernel utilization by generating a non-autoregressive draft and speeding up decoding significantly.

SELECT in _ count y

SELECT in _ count y _ tu ition _ per

SELECT in _ count y _ tu ition _ per_ credit _ ~~credit _~~

SELECT in _ count y _ tu ition _ per_ credit _ hour __ fall _ _

SELECT in _ count y _ tu ition _ per_ credit _ hour __ fall _ 2009 _ FROM table _

SELECT in _ count y _ tu ition _ per_ credit _ hour __ fall _ 2009 _ FROM table _ 22 30 88 81 _

SELECT in _ count y _ tu ition _ per_ credit _ hour __ fall _ 2009 _ FROM table _ 22 30 88 81 _ 2 WHERE college = "

SELECT in _ count y _ tu ition _ per_ credit _ hour __ fall _ 2009 _ FROM table _ 22 30 88 81 _ 2 WHERE college = " Mer ~~Er " College <\s>~~

SELECT in _ count y _ tu ition _ per_ credit _ hour __ fall _ 2009 _ FROM table _ 22 30 88 81 _ 2 WHERE college = " Mer Cer " <\s>

Figure 11. Speculative streaming on SQL generation task for $\gamma = 4$ and $k = 1$, each pass verifies the previous draft and generates a maximum of 5 tokens. For instance in pass 4, "$credit$" and "_" (shown in red) are rejected and "$hour$", "_", "$fall$", "_", "_" are speculated.

Person 2 # ~~and~~

Person 2 # thinks Lincoln ~~is a character~~

Person 2 # thinks Lincoln was a ~~character and he~~

Person 2 # thinks Lincoln was a man of ~~character and he~~

Person 2 # thinks Lincoln was a man of sound character and # person

Person 2 # thinks Lincoln was a man of sound character and # person 1 # adm ires him

Person 2 # thinks Lincoln was a man of sound character and # person 1 # adm ires him for his courage and ~~and~~

Person 2 # thinks Lincoln was a man of sound character and # person 1 # adm ires him for his courage and rights and humility . </s>

Figure 12. Speculative streaming on Dialog Summarization task for $\gamma = 4$ and $k = 1$, each pass verifies the previous draft and generates a maximum of 5 tokens. For instance, in pass 3, "is", "a", "$character$" are rejected and "was", "a", "$character$", "and", "he" are speculated.